Automated Invoice Capture - System Description

What it is

A system that reads incoming supplier invoices, presents every extracted value visually marked on the invoice for a person to confirm, codes each cost against Marathon's dimensions, and delivers the verified, coded invoice into Marathon.

The core principle is deliberate: **the software proposes, a person verifies.** It never posts a cost on its own. Every value it reads is drawn as a coloured box on the invoice so the reviewer can see at a glance where each number came from and confirm or correct it in seconds.

Marathon remains the system of record. This system is the step between an invoice arriving and a correctly coded preliminary entry existing in Marathon.

The pipeline - six stages

Stage	What happens	Status
1 - Arrival	Invoices arrive by watched folder or browser upload; a permanent copy is fingerprinted (SHA-256); duplicates are recognised, not reprocessed.	Built
2 - Reading	Each page is rendered to an image and read by a vision model (or a built-in mock).	Built
3 - Interpretation	Fields are identified with a confidence score and a bounding box tied to the exact words on the page.	Built
4 - Verification	A person confirms or corrects on a split screen; arithmetic and per-country checks flag problems first.	Built
5 - Coding	Each invoice row is coded to a Marathon project; the rows must sum to the net.	Built
6 - Handover	The verified, coded invoice is delivered to Marathon reliably and once only.	Built

Plus: a SQLite store with an append-only, tamper-evident audit trail, and a searchable, paged review queue.

How it works, stage by stage

Arrival. A background worker polls a watched folder every few seconds (robust over network shares, where file-change events are unreliable). A file is only picked up once its size has settled and it looks like a complete PDF, then it is claimed by an atomic rename. Files move through a visible state machine: `incoming -> processing -> processed`, with `duplicates` and `failed` (each failure carries a short `.error.txt`). The browser upload feeds the same pipeline.

Reading & interpretation. Pages are rendered with pypdfium2. A vision model (Claude Opus 4.8) reads each page and returns the fields - supplier, invoice number, dates, net/VAT/total, currency, VAT number, IBAN, payment reference, purchase order - each with a confidence and a bounding box. Boxes are stored normalised (0-1) so they stay aligned at any display size. When no API credential is present the system runs in **mock mode** with realistic sample data, so the whole application is usable offline.

Verification. The screen is split: the invoice on the left with colour-coded boxes, an editable form on the right,

permanently linked in both directions. Before the reviewer looks, a battery of checks has already run:

- **Arithmetic** - net + VAT = total.
- **Numbers** - locale-aware parsing (`1.234,56` vs `1,234.56` vs `1 234,56`).
- **Tax & bank identifiers** - structural VAT and IBAN validation, and a payment-reference check digit (via python-stdnum).
- **Fraud signal** - supplier country vs bank country mismatch.

Anything that fails is flagged, and the reviewer lands on the first problem.

Coding. An invoice is not one project - it carries a set of allocation lines, one per printed row, each coded to a Marathon project. The rows must sum exactly to the net total, and a mismatch is flagged.

Handover. On approval the coded invoice is delivered to Marathon using the transactional-outbox pattern: the handover is enqueued in the same database transaction as the verification, then a separate worker delivers it with retries. An idempotency key (the invoice fingerprint) guarantees the same supplier invoice can never be posted twice. Marathon's returned reference is stored against the invoice and recorded in the audit trail.

Architecture

```
Browser (React / Vite)
   |  /api
   v
FastAPI + Uvicorn --> intake worker (watched folder)
   |                  +> outbox worker (delivery to Marathon)
   +- pypdfium2 -> page images
   +- vision model (Claude Opus 4.8)  [or mock]
   +- python-stdnum (VAT / IBAN / Luhn)
   +- SQLite  (invoices, fields, line_items, audit, outbox)
   +- filesystem (original PDFs + page images)
                          |
                          v
                    Marathon adapter (API / import / Peppol)
```

- **Backend:** Python, FastAPI, Uvicorn.
- **Reading:** pypdfium2, Pillow, Claude Opus 4.8 vision (mock fallback).
- **Validation:** python-stdnum.
- **Store:** SQLite (WAL, STRICT tables) for structured data + audit; filesystem for the immutable originals and page images.
- **Frontend:** React, Vite, an SVG overlay bound to the page.

Data model (SQLite)

Table	Holds
invoices	one row per invoice: filename, status, summary (supplier/total/currency), search text, verification and handover state
fields	the extracted header fields (value, confidence, status, box)
line_items	the allocation lines (row + amount + assigned project)
audit	append-only, hash-chained history of every change
outbox	the Marathon handover queue (idempotent, with retries)

Originals (`original.pdf`) and rendered page images live on disk, not in the DB.

Key design decisions

- **SQLite as the working store.** With ~1,000 invoices/day and two reviewers, and Marathon holding the permanent record, SQLite is the simplest thing that fully does the job - no server to

run, back up or secure. The design keeps a clear path to PostgreSQL if scale ever demands it.

- **Audit trail is append-only and hash-chained.** Every event (ingest, each field correction with old->new, project coding, verification, handover) is recorded with who and when. Each row's hash includes the previous row's, so any later edit or deletion is detectable; an integrity endpoint proves it.
- **Idempotent handover.** The outbox + idempotency key mean a verified invoice cannot be lost in transit and cannot be delivered twice.
- **Mock mode.** The system runs end-to-end with no API key or credit, using realistic sample invoices - so the verification screen and full workflow can be exercised before any billing is set up.

Current status and what is not yet built

Implemented and working end-to-end (in mock mode): all six pipeline stages, the SQLite store, the audit trail, and the searchable/paged queue.

Not yet built (from the proposal):

- **Live extraction** is wired but blocked on Developer Platform API billing; the code switches from mock to live automatically once a credential with credit is present.
- **Online VIES** lookup (confirm a VAT number is *registered*, not just valid), reverse-charge recognition, and currency conversion.
- **Layout-fingerprint cache** so previously-seen suppliers are read deterministically and cheaply.
- **Object storage** (S3/MinIO), a **PDF.js** overlay, user accounts and optional four-eyes approval, and the scheduled **purge** of invoice content after confirmed handover.

Automated Invoice Capture - Mini Manual

A short guide to running the system and using the verification screen.

Running it

Two processes: the backend (API + workers) and the frontend (web UI).

Backend

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev
```

Then open **http://localhost:5173**. The dev server proxies `/api` to the backend on port 8000.

Mock vs live reading. With no API credential the backend runs in mock mode (realistic sample data), so everything works offline. Set `ANTHROPIC_API_KEY` (or sign in with the `ant` CLI) to switch to real extraction - no code change.

Sample data. To fill the queue with example invoices:

```
backend/.venv/bin/python make_samples.py 48
```

This drops 48 varied invoices into the intake folder; the intake worker ingests them within a few seconds.

Getting invoices in

Two ways, both feeding the same pipeline:

- **Watched folder** - drop a PDF into `backend/data/intake/incoming/`. Within a

few seconds it is ingested and appears in the review queue. Processed originals move to `processed/`; duplicates to `duplicates/`; unreadable files to `failed/` (with a short `.error.txt`).

- **Upload** - click **Upload invoice (PDF)** in the top bar.

Only PDFs are accepted. The same invoice arriving twice is recognised by fingerprint and not processed again.

The review queue

The home screen is the **review queue**:

- **Search** - type a supplier, invoice number, or amount fragment.
- **Tabs** - All / To review / Verified.
- **Rows** - supplier, filename, total, a **! N** badge if there are checks to

look at, and the state (to review / verified / -> Marathon).

- **Paging** - Prev / Next, 15 per page.

Click any row to open it. The queue refreshes automatically, so folder-dropped invoices appear without reloading.

The verification screen

The screen is split: the **invoice on the left** with coloured boxes, the **editable form on the right**. They are linked both ways.

What the colours mean

Colour	Meaning
Green	Read with high confidence, passed all checks
Amber	Low confidence, or a warning - please look
Red	Failed a check, or is contradictory
Grey (dashed)	Expected but not found on the page
Blue	The field you are working on now

Colour is never the only signal - each state also has a distinct line weight and dash pattern, so the screen stays usable under colour-vision deficiency.

Binding

- Click or Tab into a field -> its box lights up on the invoice, the rest dim.
- Click a box on the invoice -> the matching field is focused, ready to edit.

Keyboard

Key	Action
Tab	Move to the next field
Enter	Confirm and advance
Cmd / Ctrl + Enter	Approve the whole invoice

The signals panel (top right) lists the checks: VAT and IBAN validity, the payment-reference check digit, the net + VAT = total result, the supplier-vs-bank country fraud check, and whether the line items sum to the net. Fix anything red or amber before approving.

Coding the line items

Below the fields is the **line-items table** - one row per printed line.

- Assign a **project** to each row from the dropdown (Marathon's projects).
- One invoice can be split across several projects - code each row separately.
- The header shows coding progress (e.g. "2/3 coded").
- The rows must sum exactly to the net total; a mismatch is flagged.

Approving and handover to Marathon

Press **Approve** (or Cmd/Ctrl + Enter). This:

1. Records your confirmed values and coding in the audit trail (old -> new, by whom, when). 2. Enqueues the invoice for delivery to Marathon.

You'll see the handover status by the button:

> **Verified (ok)** - *Handing over to Marathon...* -> **Delivered to Marathon - MAR-xxxxxxxx**

Back in the queue the invoice shows a **-> Marathon** badge. Delivery retries on failure and can never post the same invoice twice.

History (audit trail)

Every open invoice has a collapsible **History** panel showing what the system read, every correction, the coding, verification, and the Marathon handover - each with who and when. The trail is append-only and tamper-evident.

Configuration (environment variables)

Variable	Purpose	Default
`ANTHROPIC_API_KEY`	Enables real extraction (else mock)	unset (mock)
`INVOICE_CAPTURE_MOCK`	Force mock mode (`1`)	off
`INVOICE_CAPTURE_LIVE`	Force live mode (`1`)	off

Variable	Purpose	Default
`INVOICE_INTAKE_DIR`	Watched-folder location	`backend/data/intake`
`INVOICE_INTAKE_POLL`	Folder poll interval (seconds)	`5`
`INVOICE_INTAKE_ENABLED`	Turn the folder worker on/off	on
`INVOICE_MAX_PAGES`	Max pages sent to the model per invoice	`8`
`MARATHON_OUTBOX_ENABLED`	Turn the handover worker on/off	on
`MARATHON_OUTBOX_POLL`	Handover poll interval (seconds)	`3`
`MARATHON_FAIL_ATTEMPTS`	Simulate N failed handover attempts (testing)	`0`
`INVOICE_DB_PATH`	SQLite database file location	`backend/data/invoices.db`

Keep the SQLite database on local disk - not a network share.

API reference (for integrators)

Endpoint	Purpose
`POST /api/invoices`	Upload a PDF
`GET /api/invoices?q=&status=&page=&page_size=`	The review queue (search + paging)
`GET /api/invoices/{id}`	Full result for the verification screen
`GET /api/invoices/{id}/pages/{n}.png`	A rendered page image
`POST /api/invoices/{id}/verify`	Save corrections + coding; enqueue handover
`GET /api/invoices/{id}/audit`	The audit trail
`GET /api/invoices/{id}/handover`	Marathon handover status
`GET /api/projects`	Marathon projects (demo list)
`GET /api/audit/integrity`	Recompute and verify the audit hash chain

Troubleshooting

- **Blank screen** - hard-reload the browser (Cmd/Ctrl + Shift + R).
- **"Extraction failed" on upload** - the API has no credit. Either add

Developer Platform credit, or run in mock mode (unset the key / set `INVOICE_CAPTURE_MOCK=1`). A Claude subscription funds Claude Code, not the API.

- **Dropped file not appearing** - give it ~5-10s (it must settle first);

check `data/intake/failed/` for a `.error.txt`.

- **Database errors** - ensure the SQLite file is on local disk, not a network

share.